

```

\documentclass[12pt]{article}
\usepackage{sbc-template}
\usepackage{graphicx,url}
\usepackage[utf8]{inputenc}
\usepackage{longtable}

\sloppy

\title{O Apache Kafka como Middleware de Eventos em Sistemas Distribuídos:
Arquitetura, Comparativos e Estudo de Caso Prático}

\author{Henrique Haag\inst{1} \\\
Arthur Wild\inst{1} }

\address{Bacharelado em Ciência da Computação\\
Universidade do Vale do Rio dos Sinos - UNISINOS\\
Rio Grande do Sul -- RS -- Brasil
\email{haaghenrique@edu.unisinos.br}
\email{wildarthur@edu.unisinos.br}
}

\begin{document}

\maketitle

\begin{abstract}
This paper analyzes Apache Kafka as an event streaming middleware under the
perspective of Distributed Systems. We examine Kafka's core architecture,
including its commit log structure, partition mechanics, and the KRaft consensus
protocol, contrasting it with low-level sockets and traditional queue-based
brokers such as RabbitMQ. Furthermore, we explore its practical application in
cloud-native core banking architectures, demonstrating its role in transactional
reliability, high-throughput replication across datacenters, and modern
microservices patterns.
\end{abstract}

\begin{resumo}
Este artigo analisa o Apache Kafka como um middleware de streaming de eventos
sob a perspectiva de Sistemas Distribuídos. São examinados os fundamentos de sua
arquitetura de log de confirmação replicado, mecanismos de partição e o
protocolo de consenso KRaft, contrastando-o com sockets de baixo nível e
corretores tradicionais de mensageria como o RabbitMQ. Adicionalmente, explora-
se sua aplicação prática na modernização de arquiteturas transacionais de core
banking nativas da nuvem, evidenciando seu papel na garantia de consistência,
replicação geográfica e implementação de padrões modernos de microsserviços.
\end{resumo}

\section{Introdução}
Nos sistemas distribuídos modernos, a comunicação eficiente, confiável e robusta
entre processos heterogêneos é um dos pilares fundamentais para garantir
escalabilidade e tolerância a falhas. Historicamente, a comunicação direta via
interfaces de baixo nível, como \textit{sockets} TCP/IP, impunha restrições
severas ao design de software corporativo. A comunicação ponto a ponto por
\textit{sockets} exige acoplamento temporal (ambos os nós precisam estar online
simultaneamente) e acoplamento espacial (os nós precisam conhecer previamente os
endereços IP e portas de rede uns dos outros). Além disso, delegar ao
desenvolvedor a gestão de buffers de memória, retransmissão de pacotes perdidos
e controle de fluxo eleva drasticamente a complexidade do sistema.

Para mitigar tais limitações, surgiram os middlewares de mensageria como camadas
de aplicação de software. Embora padrões baseados no modelo de requisição-
resposta síncrono (como APIs REST e SOAP sobre HTTP) tenham simplificado a
integração de sistemas, eles introduzem novos gargalos sob alta carga de
requisições, induzindo latência acumulada por chamadas sequenciais e aumentando

```

o risco de falhas em cascata no ecossistema de microsserviços \cite{devto:2025}.

Como resposta a esses desafios, o modelo arquitetural orientado a eventos (Event-Driven Architecture - EDA) consolidou-se ao propor o desacoplamento completo entre as entidades produtoras e consumidoras de dados \cite{devto:2025}. É nesse paradigma que o \textbf{Apache Kafka} se destaca como uma plataforma de streaming de eventos distribuídos de alta disponibilidade, redundância e altíssimo desempenho \cite{cloudera:2026}. Ao contrário de brokers de mensageria que operam com filas efêmeras de consumo destrutivo, o Kafka baseia sua infraestrutura no conceito de um log de commits imutável e replicado, persistido diretamente em disco com algoritmos de acesso em complexidade constante $O(1)$ \cite{cloudera:2026, aws:2026}. Isso permite a múltiplos consumidores independentes ler e reprocessar eventos históricos em seu próprio ritmo (\textit{replayability}) \cite{devto:2025, aws:2026}.

\section{0 Apache Kafka: Arquitetura e Funcionamento}

\label{sec:kafka}

O Apache Kafka foi concebido originalmente no LinkedIn para rastrear comportamentos de usuários em tempo real e, posteriormente, doado à \textit{Apache Software Foundation} \cite{cloudera:2026}. Ele se consolidou como um barramento de mensageria distribuído, escalável, redundante e de alto desempenho \cite{cloudera:2026}. Esta seção descreve a organização interna do middleware, seus mecanismos de consumo e sua evolução em termos de consenso distribuído sob a perspectiva de Sistemas Distribuídos.

\subsection{Modelo de Log Replicado e Estrutura Física}

Diferente de sistemas de mensageria tradicionais que utilizam filas efêmeras de consumo destrutivo, o Kafka implementa o armazenamento de dados com base em um \textit{commit log} distribuído e replicado \cite{cloudera:2026}. Os dados são estruturados em tópicos, os quais são divididos logicamente em partições \cite{cloudera:2026}. Cada partição funciona como um arquivo de log ordenado e imutável do tipo \textit{append-only}, onde cada nova mensagem é anexada sequencialmente ao final da estrutura \cite{cloudera:2026}. Internamente, os logs de cada partição são fragmentados em arquivos físicos denominados segmentos de log (\textit{log segments}), gerenciados por limites de tamanho (geralmente 1 GB) ou de tempo \cite{cloudera:2026}.

A distribuição física dessas partições ocorre entre os servidores do cluster, chamados de \textit{brokers} \cite{cloudera:2026}. Para garantir tolerância a falhas e alta disponibilidade, cada partição possui um fator de replicação associado \cite{cloudera:2026}. Dentre as réplicas de uma partição, uma é eleita como líder (\textit{leader}), enquanto as demais atuam como seguidoras (\textit{followers}) \cite{cloudera:2026}. Toda operação de escrita e leitura realizada pelos clientes (produtores e consumidores) é direcionada para a réplica líder \cite{cloudera:2026}. As réplicas seguidoras limitam-se a replicar de forma assíncrona os registros da líder para fins de redundância \cite{cloudera:2026}.

As réplicas seguidoras que conseguem acompanhar o ritmo de gravação do líder são classificadas como réplicas em sincronia (\textit{In-Sync Replicas} - ISR) \cite{cloudera:2026}. O nível de durabilidade das escritas é controlado pela configuração \texttt{acks} no produtor:

\begin{itemize}

\item \texttt{acks=0}: O produtor não aguarda confirmação do broker, enviando os dados sem garantia de recebimento para maximizar a vazão de rede \cite{cloudera:2026}.

\item \texttt{acks=1}: O líder envia a confirmação de sucesso imediatamente após anexar a mensagem ao seu log local de escrita, sem esperar a replicação das seguidoras \cite{cloudera:2026}.

\item \texttt{acks=all}: O líder aguarda que todas as réplicas ativas dentro do conjunto ISR repliquem e confirmem a escrita localmente antes de enviar a confirmação de sucesso ao produtor, garantindo resiliência total contra perda de

dados em caso de falha de infraestrutura \cite{cloudera:2026}.

\subsection{Semântica de Consumo, Offset e Paralelismo}

No Kafka, a leitura de dados ocorre de maneira não destrutiva: as mensagens permanecem persistidas em disco físico de acordo com as políticas de retenção configuradas, permitindo leituras repetidas por diferentes aplicações \cite{cloudera:2026}. O controle da leitura é descentralizado, sendo de responsabilidade do consumidor através do conceito de \textit{offset} – um índice numérico sequencial imutável que indica a posição de leitura atualizada dentro da partição de um tópico \cite{cloudera:2026}.

Para viabilizar escalabilidade horizontal, o Kafka organiza os consumidores em grupos de consumo (\textit{consumer groups}) \cite{cloudera:2026}. Dentro de um mesmo grupo, cada partição de um tópico é atribuída de forma exclusiva a, no máximo, um único consumidor ativo, o que evita duplicidade no processamento das mensagens \cite{cloudera:2026}. Para dimensionar corretamente o número de partições (\$P\$) em produção com base na vazão total desejada do sistema (\$TT\$), aplica-se a seguinte formulação teórica \cite{cloudera:2026}:

$$P = \max(N_P, N_C)$$

\label{eq:partitions}

Onde \$N_P\$ representa o número de produtores necessários, determinado pela divisão da vazão esperada do sistema pela capacidade máxima de um único produtor (\$N_P = TT / TP\$) \cite{cloudera:2026}. Da mesma forma, \$N_C\$ define o número de consumidores necessários para processar o fluxo sem atrasos, baseado no throughput máximo de consumo por partição (\$N_C = TT / TC\$) \cite{cloudera:2026}.

\subsection{Consenso e Metadados do Cluster: Do ZooKeeper ao KRaft}

Historicamente, a coordenação distribuída dos metadados do Kafka (como estados dos brokers, tópicos e líderes de partições) dependia de um cluster externo do Apache ZooKeeper \cite{cloudera:2026}. Contudo, essa arquitetura de duas camadas gerava sobrecarga de rede, atrasos na sincronização de metadados em larga escala e risco de falhas de consistência durante eleições de novos líderes \cite{cloudera:2026}.

Com a introdução do protocolo KRaft (\textit{Kafka Raft}), o gerenciamento de metadados passou a ser embutido nativamente no cluster através de um quórum de controladores (\textit{controllers}) baseados em Raft \cite{cloudera:2026}. Nesse modo, um set de brokers especializados, chamados de controladores, é implantado para formar um cluster focado na manutenção do estado de metadados \cite{cloudera:2026}. Um controlador ativo é eleito como líder do quórum de controladores para centralizar as alterações de estado, gravando-as no tópico interno de partição única \texttt{_cluster_metadata} \cite{cloudera:2026}. Os outros controladores replicam ativamente esse tópico (atuando no conjunto ISR), enquanto os brokers tradicionais de dados operam apenas como observadores (\textit{observers}), mantendo um cache local de metadados assíncrono que é atualizado periodicamente a partir das requisições de consulta (\textit{heartbeats}) enviadas ao líder \cite{cloudera:2026}.

A consistência dos metadados é mantida pela definição do \textit{High Watermark} (HWM) adaptada no KRaft: diferentemente da replicação de dados comum do Kafka, o HWM do KRaft representa o \textit{offset} mais alto que foi replicado pela maioria absoluta do quórum de controladores, seguindo rigorosamente o modelo matemático do Raft \cite{cloudera:2026}. No caso de falhas e eleição de um novo líder de metadados, o quórum vota de forma baseada em timeout utilizando o maior log ou época de cada candidato \cite{cloudera:2026}. Uma vez eleito, as réplicas seguidoras executam a reconciliação de log (\textit{log reconciliation}),

truncando seus registros de metadados divergentes até a posição demarcada pelo HWM para depois sincronizar os novos eventos ditados pelo líder \cite{cloudera:2026}.

\section{Comparação Técnica: Apache Kafka vs. RabbitMQ}

\label{sec:comparacao}

A comparação entre middlewares de mensageria assíncrona exige contrastar seus paradigmas fundamentais de armazenamento, consistência e roteamento. O RabbitMQ e o Apache Kafka nasceram em extremos opostos do problema e convergiram ao longo do tempo (KIP-932 no Kafka \cite{rabbitmq:2026} e Streams no RabbitMQ \cite{rabbitmq:2026}). A Tabela \ref{tab:kafka_vs_rabbitmq} sintetiza as diferenças técnicas estruturais.

\begin{table*}[htbp] \centering \caption{Comparação Técnica Estrutural: Apache Kafka vs. RabbitMQ} \label{tab:kafka_vs_rabbitmq} \begin{tabular}{ p{3.2cm} p{6.0cm} p{6.0cm} } \hline \textbf{Critério Técnico} & \textbf{Apache Kafka} & \textbf{RabbitMQ} \\ \hline \textbf{Paradigma Principal} & Log de commits distribuído e imutável \cite{rabbitmq:2026}. & Mensageria inteligente baseada em \textit{Exchanges}, \textit{Bindings} e Filas \cite{rabbitmq:2026}. \\ \hline \textbf{Estrutura Física} & Tópico dividido em partições distribuídas e replicadas em disco \cite{cloudera:2026}. & Filas tradicionais destrutivas ou logs \textit{append-only} (\textit{Streams}) \cite{rabbitmq:2026}. \\ \hline \textbf{Unidade de Armazenamento} & Posição sequencial (\textit{offset}) dentro de um log de partição compartilhado \cite{cloudera:2026}. & Envelope de mensagem individual contendo metadados e corpo opaco \cite{rabbitmq:2026}. \\ \hline \textbf{Roteamento de Dados} & No cliente (\textit{client-side}): o produtor escolhe a partição via chave hash; broker é agnóstico \cite{rabbitmq:2026}. & No servidor (\textit{broker-side}): as \textit{Exchanges} aplicam regras de associação (\textit{Bindings}) \cite{rabbitmq:2026}. \\ \hline \textbf{Garantia de Durabilidade} & Baseia-se em replicação ativa (ISR). Desabilita o \textit{fsync} síncrono por padrão de escrita \cite{cloudera:2026, rabbitmq:2026}. & \textbf{Quorum Queues}: replicadas via Raft, exigem gravação física e \textit{fsync} síncrono da maioria antes do confirm \cite{rabbitmq:2026}. \\ \hline \textbf{Semântica de Consumo} & Paralelismo limitado pelas partições via \textit{Consumer Groups} ou leitura concorrente via \textit{Share Groups} \cite{cloudera:2026, rabbitmq:2026}. & Consumidores concorrentes e competitivos que disputam tarefas de uma mesma fila \cite{rabbitmq:2026}. \\ \hline \textbf{Filtragem de Dados} & No cliente (\textit{client-side}): o consumidor lê toda a partição para aplicar filtros localmente \cite{rabbitmq:2026}. & No servidor (\textit{server-side}): filtros de Bloom ou expressões SQL no \textit{broker} \cite{rabbitmq:2026}. \\ \hline\end{table*}

<code>\textbf{Controle de Mensagens}</code> & Baseado em políticas de retenção globais aplicadas ao tópico inteiro <code>\cite{cloudera:2026, rabbitmq:2026}</code>. & <code>\textbf{Toolkit individual}</code>: TTL, prioridades, atrasos de entrega, retentativa com <code>\textit{backoff}</code> e <code>\textit{dead-lettering}</code> <code>\cite{rabbitmq:2026}</code>. \\ \hline <code>\textbf{Protocolos Suportados}</code> & Protocolo binário proprietário executado sobre conexões TCP nativas <code>\cite{rabbitmq:2026}</code>. & Múltiplos protocolos: AMQP 1.0, AMQP 0-9-1, MQTT, STOMP e transportes WebSocket <code>\cite{rabbitmq:2026}</code>. \\ \hline <code>\textbf{Consenso e Metadados}</code> & Quórum de controladores interno unificado via protocolo de consenso <code>\textbf{KRaft}</code> <code>\cite{cloudera:2026}</code>. & Gerenciamento do estado do <code>\textit{cluster}</code> centralizado internamente via <code>\textbf{Khepri}</code> (Raft) <code>\cite{rabbitmq:2026}</code>. \\ \hline

`\end{tabular}`
`\end{table*}`

Além do detalhamento exposto na tabela, a principal distinção operacional reside no acoplamento de roteamento. Enquanto o RabbitMQ permite aos clientes redefinir topologias de comunicação de forma dinâmica em tempo de execução via `\textit{bindings}` e `\textit{exchanges}` `\cite{rabbitmq:2026}`, o Kafka impõe a rigidez do particionamento definido no momento da gravação pelo cliente produtor `\cite{cloudera:2026}`. Adicionalmente, as garantias de durabilidade divergem significativamente: o compromisso do Kafka reside em replicação em memória ativa (page cache) sem garantia física de escrita síncrona em disco (`\textit{fsync}`) no caminho crítico de gravação, o que pode incorrer em perda de dados em cenários de queda catastrófica simultânea de energia de todas as réplicas ISR `\cite{rabbitmq:2026}`. As `\textit{Quorum Queues}` do RabbitMQ, por outro lado, garantem conformidade transacional total realizando `\textit{fsync}` síncrono consolidado por lote via WAL compartilhado entre múltiplos quóruns Raft antes de emitir a confirmação de escrita (`\textit{publisher confirm}`) `\cite{rabbitmq:2026}`.

`\section{Estudo de Caso Prático: Modernização de Core Banking Nativo da Nuvem}` `\label{sec:casodesuso}`

Os sistemas de `\textit{core banking}` representam o barramento transacional de missão crítica de instituições financeiras, englobando desde transações de depósitos e saques até fluxos de conformidade e KYC (`\textit{Know Your Customer}`) `\cite{busko:2023}`. Historicamente operados sobre `\textit{mainframes}` legados com base em Cobol e IBM DB2, as empresas buscam modernizar essas infraestruturas por razões de custo, agilidade de mercado e restrições de escalabilidade física `\cite{busko:2023}`.

`\subsection{Cargas de Trabalho Transacionais vs. Analíticas e Requisitos de Resiliência}`

Em uma plataforma bancária moderna, as cargas de trabalho transacionais e analíticas possuem características de desempenho e acordos de nível de serviço (SLA) fundamentalmente distintos `\cite{busko:2023}`. Atividades analíticas toleram janelas de indisponibilidade e metas de RPO maiores do que zero `\cite{busko:2023}`. Já as cargas transacionais exigem RPO estritamente igual a zero e RTO o mais próximo possível de zero `\cite{busko:2023}`.

Para atender a esse SLA em produção, o Apache Kafka é implantado em arquiteturas altamente resilientes conhecidas como `\textit{Stretch Clusters}` `\cite{cloudera:2026}`. Essa abordagem distribui de forma síncrona um único cluster lógico do Kafka entre datacenters geográficos distintos ou zonas de disponibilidade com latência máxima recomendada de 50 ms `\cite{cloudera:2026}`. O mecanismo utiliza a configuração de `\textit{rack awareness}` através do parâmetro `\texttt{broker.rack}` para instruir o Kafka a distribuir as partições fisicamente

entre diferentes datacenters \cite{cloudera:2026}. Produtores operando sobre transações críticas utilizam obrigatoriamente \texttt{acks=all}, garantindo que a escrita só seja validada após a replicação síncrona no page cache de todas as réplicas que compõem o ISR \cite{cloudera:2026, rabbitmq:2026}. Em conjunto com a definição rigorosa de \texttt{min.insync.replicas}, assegura-se matematicamente que ao menos um datacenter independente gravou a informação de forma íntegra antes da conclusão da transação \cite{cloudera:2026}.

\subsection{Modelos de Sucesso em Core Banking Moderno}

Três grandes implementações demonstram o impacto prático dessa arquitetura:

\begin{itemize}

\item \textbf{Thought Machine (Vault)}: Sistema operacional de core banking nativo de nuvem operando transações 24/7 com perda zero de dados \cite{busko:2023}. O Vault utiliza o processamento de eventos do Kafka em seu coração para habilitar processamento assíncrono e paralelização em escala, estabelecendo padrões para garantir integridade operacional nos fluxos paralelos \cite{busko:2023}.

\item \textbf{10x Banking}: Plataforma baseada em design orientado a domínios que adota o Kafka como seu barramento central de dados analíticos e transacionais \cite{busko:2023}. O 10x Banking emprega a arquitetura Kappa combinada com Apache Pinot e Trino para consolidar fluxos em tempo real e em lote em um único pipeline de dados, reduzindo drasticamente o tempo de mercado para novos produtos bancários personalizados \cite{busko:2023}.

\item \textbf{Custodigit}: Plataforma voltada a investimentos de ativos digitais e criptomoedas em ambiente severamente regulado \cite{busko:2023}. Utiliza o Kafka como sistema nervoso central de sua arquitetura de microsserviços, coordenando a coreografia de processos com estado por meio de microsserviços desenvolvidos com a API do \texttt{Kafka Streams} e aplicando o padrão transacional distribuído \texttt{Saga} sob semântica exatamente-uma-vez \cite{busko:2023}.

\end{itemize}

O redirecionamento de consultas e escritas de mainframes legados para o Apache Kafka (\texttt{mainframe offloading}) proporciona economias significativas de custos operacionais. O Royal Bank of Canada (RBC) economizou milhões de dólares em despesas fixas (OPEX) ao descarregar logs de transações de mainframe para o Kafka, viabilizando novos aplicativos orientados a eventos sem onerar as máquinas legadas centrais \cite{busko:2023}.

\section{Conclusão}

\label{sec:conclusao}

A análise do Apache Kafka sob a ótica de Sistemas Distribuídos consolida a transição dos modelos de comunicação síncronos ponto a ponto (sockets) para arquiteturas orientadas a eventos em larga escala \cite{cloudera:2026, devto:2025}. Ao abstrair a comunicação direta de transporte por sockets em favor de um log distribuído e persistente com complexidade $O(1)$ \cite{cloudera:2026}, o Kafka remove os acoplamentos espaciais e temporais do caminho crítico de execução das aplicações \cite{devto:2025, aws:2026}.

Contudo, a escolha do middleware exige avaliar compensações técnicas fundamentais. O Kafka delega maior responsabilidade de roteamento e consistência às aplicações clientes \cite{cloudera:2026, rabbitmq:2026}. A sua resiliência transacional fundamenta-se estritamente na redundância do cluster e na eficiência da replicação ativa via page cache das ISRs, sem persistência física síncrona (\texttt{fsync}) individualizada de mensagens \cite{rabbitmq:2026}.

Dessa forma, o Apache Kafka consolida-se como o padrão de engenharia quando os requisitos do sistema demandam processamento contínuo de dados de alto throughput, semântica exatamente-uma-vez, replay de logs históricos e processamento stateful em tempo real \cite{cloudera:2026, busko:2023}. Se o cenário demandar roteamento inteligente de mensagens no servidor, latência sob controle individual fino de fila ou durabilidade de mensagem persistida sob

flush físico imediato no disco de cada nó, middlewares com design de fila tradicional, tais como o RabbitMQ, permanecem recomendados \cite{rabbitmq:2026}.

```
\bibliographystyle{sbc}
\begin{thebibliography}{9}
```

```
\bibitem{cloudera:2026}
Cloudera Inc. (2026).
\newblock Apache Kafka Overview.
\newblock {\em Cloudera Runtime 7.3.2 Documentation}.
```

```
\bibitem{devto:2025}
Holigi, C. (2025).
\newblock Building Event-Driven Microservices with Apache Kafka: A Practical
Architecture for High-Scale Platforms.
\newblock {\em DEV Community}.
```

```
\bibitem{itsolutionss:2024}
IT Solutionss (2024).
\newblock Curso Apache Kafka com Kubernetes e Docker.
\newblock {\em IT Solutionss Treinamentos}.
```

```
\bibitem{busko:2023}
Busko, P. (2023).
\newblock Modernização do Core Banking nativo da nuvem com Apache Kafka.
\newblock {\em Blog Pedro Busko - Streaming de Dados}.
```

```
\bibitem{opservices:2026}
Tebaldi, P. (2026).
\newblock Monitoramento de Kafka: métricas, ferramentas e alertas.
\newblock {\em OpServices Blog}.
```

```
\bibitem{aws:2026}
Amazon Web Services (2026).
\newblock O que é o Kafka? – Explicação sobre o Apache Kafka.
\newblock {\em AWS Cloud Concepts}.
```

```
\bibitem{apache:2026}
Apache Software Foundation (2026).
\newblock Overview | Apache Kafka.
\newblock {\em Apache Kafka Official Documentation}.
```

```
\bibitem{rabbitmq:2026}
Broadcom Inc. (2026).
\newblock RabbitMQ vs. Apache Kafka.
\newblock {\em RabbitMQ Official Documentation}.
```

```
\bibitem{cetax:2026}
Cetax (2026).
\newblock Tendências da Engenharia de Dados para 2026: IA, Lakehouse, Data Mesh
e o Futuro dos Dados.
\newblock {\em Cetax Blog}.
```

```
\end{thebibliography}
```

```
\end{document}
```